

Kernel PCA

ML A-Z — Part 11

1 Motivation

Standard PCA (Part 10) finds directions of maximum variance by eigendecomposing the covariance matrix $\Sigma = \frac{1}{n}X^\top X$. Because Σ is built from *linear* combinations of the original features, the components it finds are linear subspaces — it can only "see" variance along straight lines through the data. If the true structure in the data is nonlinear (e.g. classes separated by a curved boundary, or lying on a manifold like concentric circles), no linear projection will reveal it, no matter how many components are kept.

Kernel PCA fixes this by first mapping the data into a much higher-dimensional (possibly infinite-dimensional) feature space via a nonlinear map ϕ , where the structure becomes linear, and then running ordinary PCA in that space. The obstacle is that ϕ can be expensive or impossible to compute explicitly — the **kernel trick** sidesteps this entirely.

2 The Kernel Trick

2.1 Idea

Many linear algorithms, PCA included, only ever need the data through inner products $\mathbf{x}_i^\top \mathbf{x}_j$, never through the raw feature vectors themselves. The kernel trick observes that if an algorithm can be written purely in terms of such inner products, then to run it in a feature space $\phi(\mathbf{x})$ we don't need ϕ explicitly — we only need a function

$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

that computes the inner product *in that space directly from the original vectors*. Replacing every $\mathbf{x}_i^\top \mathbf{x}_j$ with $k(\mathbf{x}_i, \mathbf{x}_j)$ implicitly performs the algorithm in the (possibly infinite-dimensional) space of ϕ , at the cost of evaluating k instead of an explicit dot product — often $O(p)$ work either way, regardless of how large the target space is.

2.2 Common Kernels

Kernel	Formula	Notes
Linear	$k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$	Recovers ordinary PCA
Polynomial	$k(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x}^\top \mathbf{y} + c)^d$	Captures feature interactions up to degree d
RBF (Gaussian)	$k(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \ \mathbf{x} - \mathbf{y}\ ^2)$	Infinite-dimensional feature space; most common default
Sigmoid	$k(\mathbf{x}, \mathbf{y}) = \tanh(\gamma \mathbf{x}^\top \mathbf{y} + c)$	Behaves like a neural-net activation

A function k is a valid kernel — i.e. actually corresponds to an inner product in *some* feature space — iff it is symmetric and positive semi-definite (**Mercer's condition**): for any finite set of points $\mathbf{x}_1, \dots, \mathbf{x}_n$, the Gram matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ must be PSD.

3 From PCA to Kernel PCA

3.1 Ordinary PCA, Rewritten with Inner Products

Assume the data is mean-centered in feature space. Ordinary PCA diagonalizes $\Sigma = \frac{1}{n}X^\top X \in \mathbb{R}^{p \times p}$. It turns out the same eigenvectors can be recovered from the $n \times n$ Gram matrix $XX^\top$ instead: if $\mathbf{v}$ is a (unit) eigenvector of Σ with eigenvalue λ , then $\mathbf{v} = \frac{1}{\sqrt{n\lambda}}X^\top \mathbf{a}$ for some eigenvector $\mathbf{a}$ of $XX^\top$ with the same eigenvalue (up to a factor of n), i.e. every principal direction is expressible as a linear combination of the (centered) data points themselves. This is the key fact that lets PCA be reformulated using only pairwise inner products $XX^\top$, without ever referring to Σ or the raw feature space directly.

3.2 Kernelizing

Replace every inner product $\mathbf{x}_i^\top \mathbf{x}_j$ with $k(\mathbf{x}_i, \mathbf{x}_j)$, giving the $n \times n$ kernel (Gram) matrix K with $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$. Because ϕ is never computed explicitly, K must also be **centered in feature space** without access to the (unknown) feature-space mean:

$$\tilde{K} = K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n,$$

where $\mathbf{1}_n$ is the $n \times n$ matrix with every entry $1/n$. This identity centers K exactly as if $\phi(\mathbf{x}_i)$ had been mean-subtracted before forming the Gram matrix.

Eigendecomposing $\tilde{K}$ gives eigenvalues $\lambda_1 \geq \dots \geq \lambda_n \geq 0$ and eigenvectors $\mathbf{a}_1, \dots, \mathbf{a}_n$. The projection of point $\mathbf{x}$ onto the i -th kernel principal component is then

$$z_i(\mathbf{x}) = \sum_{j=1}^n a_{ij} k(\mathbf{x}_j, \mathbf{x}),$$

i.e. a weighted sum of kernel evaluations against the training points, normalized so that $\|\mathbf{v}_i\| = 1$ in feature space (equivalent to scaling $\mathbf{a}_i$ by $1/\sqrt{\lambda_i}$). Note there is no explicit $\mathbf{v}_i$ to compute or store — the component only ever appears through kernel evaluations, which is exactly what makes the method tractable in an infinite-dimensional space.

3.3 Algorithm

Algorithm 1 Kernel PCA

- 1: Choose a kernel k (and its hyperparameters, e.g. γ for RBF)
 - 2: Compute the Gram matrix $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ for all training pairs
 - 3: Center K in feature space: $\tilde{K} = K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n$
 - 4: Compute eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$ and eigenvectors $\mathbf{a}_1, \dots, \mathbf{a}_n$ of $\tilde{K}$
 - 5: Normalize each $\mathbf{a}_i \leftarrow \mathbf{a}_i / \sqrt{\lambda_i}$
 - 6: Choose k components; project new points via $z_i(\mathbf{x}) = \sum_j a_{ij} k(\mathbf{x}_j, \mathbf{x})$
-

4 PCA vs. Kernel PCA

	PCA	Kernel PCA
Structure captured	Linear	Nonlinear (via ϕ)
Matrix diagonalized	$\Sigma \in \mathbb{R}^{p \times p}$ (covariance)	$\tilde{K} \in \mathbb{R}^{n \times n}$ (centered Gram)
Cost driver	Number of features p	Number of training points n
Components stored as	Vectors $\mathbf{v}_i \in \mathbb{R}^p$	Weighted combinations of training points
Inverse transform	Exact (linear)	Only approximate (pre-image problem)

Because Kernel PCA diagonalizes an $n \times n$ matrix rather than a $p \times p$ one, it scales with the number of *samples*, not features — the opposite of ordinary PCA, and a relevant consideration for large training sets.

In the accompanying notebook, Kernel PCA with an RBF kernel is applied to the same Wine dataset used for PCA/LDA in Part 10 (13 features, 3 cultivar classes), reducing to 2 components before logistic regression. It reaches the same perfect test accuracy that LDA did, despite being unsupervised like PCA — here the nonlinear RBF mapping is expressive enough to make the classes linearly separable in the transformed space, something linear PCA could not guarantee.

5 Key Takeaways

- Kernel PCA applies the kernel trick to PCA: it implicitly maps data into a higher-dimensional feature space via ϕ and finds principal components there, without ever computing ϕ explicitly.
- The trick works because PCA can be rewritten entirely in terms of pairwise inner products, which the kernel function $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ supplies directly.
- The RBF kernel is the most common default and corresponds to an infinite-dimensional feature space; the choice of kernel and its hyperparameters (e.g. γ) controls what kind of nonlinear structure can be captured.
- Unlike PCA, Kernel PCA's cost scales with the number of training points n (via the $n \times n$ Gram matrix), not the number of features p , and its components have no clean linear form — they only exist as weighted sums of kernel evaluations against the training data.